

3.7.4 Somador de Ponto Flutuante Simplificado

Tradução Técnica e Documentação de Projeto em VHDL

Material extraído de literatura de Projeto de Sistemas Digitais | Projetos em VHDL

O ponto flutuante é outro formato para representar números. Com a mesma quantidade de bits, o alcance dinâmico no formato de ponto flutuante é muito maior do que no formato de número inteiro com sinal. Embora a linguagem VHDL possua um tipo de dado de ponto flutuante nativo, ele é excessivamente complexo para ser sintetizado automaticamente em hardware.

Exemplos do Processo de Adição

A Tabela 1 ilustra o processo passo a passo da adição em ponto flutuante utilizando representação decimal simplificada para facilitar a compreensão do algoritmo.

Exemplo	Entradas	1. Ordenar (Sort)	2. Alinhar (Align)	3. Somar/Subtrair	4. Normalizar
Ex. 1	+0.54E3 -0.87E4	-0.87E4 +0.54E3	-0.87E4 +0.05E4	-0.82E4	-0.82E4
Ex. 2	+0.54E3 -0.55E3	-0.55E3 +0.54E3	-0.55E3 +0.54E3	-0.01E3	-0.10E2
Ex. 3	+0.54E0 -0.55E0	-0.55E0 +0.54E0	-0.55E0 +0.54E0	-0.01E0	-0.00E0 (Zero)
Ex. 4	+0.56E3 +0.52E3	+0.56E3 +0.52E3	+0.56E3 +0.52E3	+1.08E3	+0.10E4

Tabela 1: Exemplos de adição em ponto flutuante.

Formato e Suposições da Representação

Uma discussão detalhada da representação em ponto flutuante está fora do escopo deste texto. Utilizamos um formato simplificado de 13 bits neste exemplo e ignoramos o erro de arredondamento. A representação é constituída por:

- Um bit de sinal, *s*, que indica o sinal do número (**1** para números negativos);
- Um campo de expoente de 4 bits, *e*, que representa o expoente;
- Um campo de significando (mantissa) de 8 bits, *f*, que representa a fração ou significando.

Valor em ponto flutuante = $(-1)^s \times 0.f \times 2^e$

Nesta formulação, $0.f \times 2^e$ representa a magnitude do número e $(-1)^s$ é apenas uma maneira formal de expressar que "*s* = 1 implica um número negativo". Como o bit de sinal é separado do restante do número, a representação em ponto flutuante pode ser considerada uma variação do formato sinal-magnitude.

Premissas do Projeto:

- Ambos os campos de expoente e significando estão em formato sem sinal (*unsigned*).

- A representação deve ser normalizada ou igual a zero. Uma representação normalizada significa que o bit mais significativo (MSB) do campo do significando deve ser '1'. Se a magnitude do resultado da computação for menor do que a menor magnitude não nula normalizada ($0.10000000 \times 2^{0000}$), ela deve ser convertida para zero.

Sob essas premissas, a maior e a menor magnitude não nula são $0.11111111 \times 2^{1111}$ e $0.10000000 \times 2^{0000}$, respectivamente, e a faixa dinâmica é de aproximadamente 2^{16} (ou seja, de $0.10000000 \times 2^{0000}$ a $0.11111111 \times 2^{1111}$).

Etapas do Algoritmo de Adição

O projeto do somador em ponto flutuante segue o processo manual de adição de números em notação científica. O cálculo é realizado em quatro etapas principais:

1. **Ordenação (Sorting):** Posiciona o número de maior magnitude no topo e o de menor magnitude na parte inferior (denominados "número grande" e "número pequeno").
2. **Alinhamento (Alignment):** Alinha os dois números para que compartilhem o mesmo expoente. Isso é feito ajustando o expoente do número pequeno para corresponder ao do número grande. O significando do número pequeno é deslocado para a direita de acordo com a diferença entre os expoentes.
3. **Adição/Subtração (Addition/Subtraction):** Soma ou subtrai os significandos dos dois números alinhados.
4. **Normalização (Normalization):** Ajusta o resultado final para o formato normalizado.

Três procedimentos de normalização podem ser necessários:

- Após uma subtração, o resultado pode conter zeros à esquerda (como no Exemplo 2).
- Após uma subtração, o resultado pode ser pequeno demais para ser normalizado, devendo ser convertido para zero (como no Exemplo 3).
- Após uma adição, o resultado pode gerar um bit de estouro/vai-um (*carry-out*) (como no Exemplo 4).

Implementação em VHDL do Somador

O somador em ponto flutuante binário utiliza um algoritmo similar. Para simplificar a implementação, o arredondamento é ignorado; durante o alinhamento e a normalização, os bits menos significativos do significando são descartados ao serem deslocados. O circuito é dividido em quatro estágios. Os sufixos 'b', 's', 'a', 'r' e 'n' nos nomes dos sinais indicam, respectivamente: *big number* (número grande), *small number* (número pequeno), *aligned number* (número alinhado), *result* (resultado da soma/subtração) e *normalized number* (número normalizado).

Listagem 3.19: Somador de Ponto Flutuante Simplificado em VHDL (fp_adder.vhd)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity fp_adder is
  port(
    sign1, sign2 : in  std_logic;
    exp1, exp2   : in  std_logic_vector(3 downto 0);
    frac1, frac2 : in  std_logic_vector(7 downto 0);
    sign_out     : out std_logic;
    exp_out      : out std_logic_vector(3 downto 0);
    frac_out     : out std_logic_vector(7 downto 0)
  );
end fp_adder;

architecture arch of fp_adder is
```

```

-- Sufixos b, s, a, n para número grande (big), pequeno (small), alinhado (aligned) e normalizado
signal signb, signs : std_logic;
signal expb, exps, expn : unsigned(3 downto 0);
signal fracb, fracs, fracn : unsigned(7 downto 0);
signal sum_norm : unsigned(7 downto 0);
signal exp_diff : unsigned(3 downto 0);
signal sum : unsigned(8 downto 0); -- 1 bit extra para o carry (vai-um)
signal lead0 : unsigned(2 downto 0);
begin

-- 1º Estágio: ordenar para identificar o maior número
process(sign1, sign2, exp1, exp2, frac1, frac2)
begin
    if (exp1 & frac1) > (exp2 & frac2) then
        signb <= sign1;
        signs <= sign2;
        expb <= unsigned(exp1);
        exps <= unsigned(exp2);
        fracb <= unsigned(frac1);
        fracs <= unsigned(frac2);
    else
        signb <= sign2;
        signs <= sign1;
        expb <= unsigned(exp2);
        exps <= unsigned(exp1);
        fracb <= unsigned(frac2);
        fracs <= unsigned(frac1);
    end if;
end process;

-- 2º Estágio: alinhar o menor número
exp_diff <= expb - exps;
with exp_diff select
    fracn <=
        fracs
            when "0000",
        "0" & fracs(7 downto 1) when "0001",
        "00" & fracs(7 downto 2) when "0010",
        "000" & fracs(7 downto 3) when "0011",
        "0000" & fracs(7 downto 4) when "0100",
        "00000" & fracs(7 downto 5) when "0101",
        "000000" & fracs(7 downto 6) when "0110",
        "0000000" & fracs(7) when "0111",
        "00000000" when others;

-- 3º Estágio: somar / subtrair
sum <= ('0' & fracb) + ('0' & fracn) when signb = signs else
        ('0' & fracb) - ('0' & fracn);

-- 4º Estágio: normalizar
-- Contar zeros à esquerda
lead0 <=
    "000" when sum(7) = '1' else
    "001" when sum(6) = '1' else
    "010" when sum(5) = '1' else
    "011" when sum(4) = '1' else
    "100" when sum(3) = '1' else
    "101" when sum(2) = '1' else
    "110" when sum(1) = '1' else
    "111";

-- Deslocar o significando conforme a contagem de zeros
with lead0 select
    sum_norm <=
        sum(7 downto 0) when "000",
        sum(6 downto 0) & '0' when "001",
        sum(5 downto 0) & "00" when "010",
        sum(4 downto 0) & "000" when "011",
        sum(3 downto 0) & "0000" when "100",

```

```

sum(2 downto 0) & "00000" when "101",
sum(1 downto 0) & "000000" when "110",
sum(0) & "0000000" when others;

-- Normalização sob condições especiais
process(sum, sum_norm, expb, lead0)
begin
  if sum(8) = '1' then
    -- Com carry out: incrementa expoente e desloca frac para a direita
    expn <= expb + 1;
    fracn <= sum(8 downto 1);
  elsif (lead0 > expb) then
    -- Resultado muito pequeno para normalizar: zera o valor
    expn <= (others => '0');
    fracn <= (others => '0');
  else
    expn <= expb - lead0;
    fracn <= sum_norm;
  end if;
end process;

-- Atribuição das saídas
sign_out <= signb;
exp_out <= std_logic_vector(expn);
frac_out <= std_logic_vector(fracn);

end arch;

```

Descrição Detalhada do Hardware por Estágio

1º Estágio: O circuito compara as magnitudes e atribui o maior número aos sinais `signb`, `expb` e `fracb`, e o menor aos sinais `signs`, `exps` e `fracs`. A comparação é feita através da concatenação `exp1 & frac1` e `exp2 & frac2`, garantindo que os expoentes sejam comparados primeiro e, em caso de igualdade, os significandos sejam avaliados.

2º Estágio: O circuito executa o alinhamento calculando primeiramente a diferença de expoentes (`expb - exps`). Em seguida, desloca o significando `fracs` para a direita por este valor, produzindo o significando alinhado `fracn`.

3º Estágio: O circuito realiza a adição/subtração em sinal-magnitude. Os operandos são estendidos em 1 bit à esquerda para acomodar o bit de vai-um (*carry-out*).

4º Estágio: O circuito realiza a normalização em três segmentos:

- O primeiro segmento conta os zeros à esquerda (atuando como um codificador de prioridade);
- O segundo segmento desloca o significando para a esquerda com base na contagem de zeros;
- O terceiro segmento verifica as condições de *carry-out* e de valor nulo, gerando a saída normalizada final.

Circuito de Teste

O somador de ponto flutuante possui dois operandos de entrada de 13 bits (totalizando 26 bits). Como a placa de prototipagem possui apenas 8 chaves (dip-switches) e 4 botões de pressão (pushbuttons), não há conectores físicos suficientes para todas as entradas.

Para viabilizar o teste, criamos um circuito de teste que fixa um operando como constante e associa os sinais de chaves duplicados ao outro operando. O resultado da soma é enviado para decodificadores hexadecimais e para o circuito de sinal, sendo exibido no display de LED de 7 segmentos.

Listagem 3.20: Circuito de Teste do Somador de Ponto Flutuante em VHDL (fp_adder_test.vhd)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity fp_adder_test is
  port(
    clk : in std_logic;
    sw   : in std_logic_vector(7 downto 0);
    btn  : in std_logic_vector(3 downto 0);
    an   : out std_logic_vector(3 downto 0);
    sseg : out std_logic_vector(7 downto 0)
  );
end fp_adder_test;

architecture arch of fp_adder_test is
  signal sign1, sign2 : std_logic;
  signal exp1, exp2   : std_logic_vector(3 downto 0);
  signal frac1, frac2 : std_logic_vector(7 downto 0);
  signal sign_out     : std_logic;
  signal exp_out      : std_logic_vector(3 downto 0);
  signal frac_out     : std_logic_vector(7 downto 0);
  signal led3, led2, led1, led0 : std_logic_vector(7 downto 0);
begin

  -- Configuração dos sinais de entrada do somador FP
  sign1 <= '0';
  exp1  <= "1000";
  frac1 <= '1' & sw(1) & sw(0) & "10101";
  sign2 <= sw(7);
  exp2  <= btn;
  frac2 <= '1' & sw(6 downto 0);

  -- Instanciação do somador FP
  fp_add_unit: entity work.fp_adder
    port map(
      sign1 => sign1, sign2 => sign2, exp1 => exp1, exp2 => exp2,
      frac1 => frac1, frac2 => frac2,
      sign_out => sign_out, exp_out => exp_out,
      frac_out => frac_out
    );

  -- Instanciação dos decodificadores hexadecimais para display de 7 segmentos
  -- Expoente
  sseg_unit_0: entity work.hex_to_sseg
    port map(hex => exp_out, dp => '0', sseg => led0);

  -- 4 LSBs do significando (fração)
  sseg_unit_1: entity work.hex_to_sseg
    port map(hex => frac_out(3 downto 0), dp => '1', sseg => led1);

  -- 4 MSBs do significando (fração)
  sseg_unit_2: entity work.hex_to_sseg
    port map(hex => frac_out(7 downto 4), dp => '0', sseg => led2);

  -- Exibição do sinal (+ / -)
  led3 <= "11111110" when sign_out = '1' else -- Traço central (sinal negativo)
          "11111111";                       -- Apagado (sinal positivo)

  -- Instanciação do módulo de multiplexação temporal do display de 7 segmentos
  disp_unit: entity work.disp_mux
    port map(
      clk => clk, reset => '0',
      in0 => led0, in1 => led1, in2 => led2, in3 => led3,
      an => an, sseg => sseg
    );
end;
```

```
end arch;
```